ENGINEERING REPORT

Vehicle Location Data Product

From a mock vehicle-location signal to queryable velocity and acceleration. Four modules, each shown here as a black box first, then opened one module at a time. Source state read on 27 Oct 2026.

Module 1 · Generator
Built and merged into main, without ground truth.
`main (change 608136a)`

Module 2 · ETL + Loader
Built; main merged into its branch.
`worktree-module2-loader-etl @ a182575`

Module 3 · Use case
Committed on its branch, not in main yet.
`module3-use-case @ c104c21`

Module 4 · BI
Committed on its branch, not in main yet.
`module4-bi @ 79c9f52`

What the project does

The product turns a vehicle-location signal into motion data that people can query. The signal describes the host vehicle as a 1 m cube of 1,000,000 points (100 per side, 1 cm apart), captured 16 times a second. No real recordings exist for the exercise, so the project builds its own source and everything after it.

Four modules do the work. Each one reads what the previous one wrote and passes on a documented set of files.

1 · Generator
Turns a short scenario (a start position and a list of move and stop segments) into a drive: a point cloud for every frame and a manifest with the hash of each file.

2 · ETL + Loader
Checks the hashes, tests every frame against six error codes (VL-001 to VL-006), keeps 9 key points per frame (8 corners and the centre), and writes them to a Parquet lake. Bad frames go to a quarantine table.

3 · Use case
Computes the velocity and acceleration of the vehicle from the centre point, and stores them as a motion table.

4 · BI
One Streamlit app with four tabs: generate a drive, inspect its data quality, see the motion, and run SQL on the lake.

The implemented use case is ego-motion extraction: the velocity and acceleration of a vehicle that drives forwards, drives backwards, stops, and changes speed. Five more use cases are written up in `docs/USE_CASES.md` and not built.

Engineering concepts and assumptions

Tags: DECISION a choice we made and can defend, ASSUMPTION something we took as given, MEASURED a number from a real run.

HOW THE SYSTEM IS CUT

One-way dependencies. DECISION
A module uses only the shared core and the modules before it. Each can be tested alone.

Files are the contract. DECISION
Module 1 hands over a drive folder (manifest schema version 2). Module 2 hands over lake tables. Each handover is a documented Parquet schema, so separate sessions built the modules in parallel.

Python SDK and SQL, no web service. DECISION
VLClient reads lake tables as Arrow and runs SQL with DuckDB. A REST API is listed as coming soon.

DATA AND IDENTITY

Scenario hash names the drive. DECISION
SHA-256 of the canonical scenario, including cube size and density. The first 12 hex characters are the folder name. The manifest keeps all 64 and the hash of every file. Same scenario, same folder.

Re-ingest is safe to repeat. DECISION
The lake is Hive-style Parquet, partitioned by drive. A drive is replaced by writing a new folder and renaming it into place. That is two renames, not one atomic step, and the docs say so.

ASSUMPTIONS ABOUT THE SIGNAL

Straight line along x. ASSUMPTION
Signed speed, no turning, y and z constant. The shape of a cube shows no orientation, so yaw could not be read back from the points.

No sensor noise. ASSUMPTION
The only error is rounding the points to whole centimetres.

Fixed point order. ASSUMPTION
Module 2 picks corners and centre by index. The corner spacing also serves as a rigid-shape check (VL-004).

Full density. DECISION
1,000,000 points per frame is 12,000,000 bytes (computed). Frames are written in batches of about 50 with zstd. The 1,000-frame demo is 287,134,433 bytes with zstd and 4,024,944,648 with snappy (measured). A scenario may have at most 2,000 frames.

ALGORITHM AND VERIFICATION

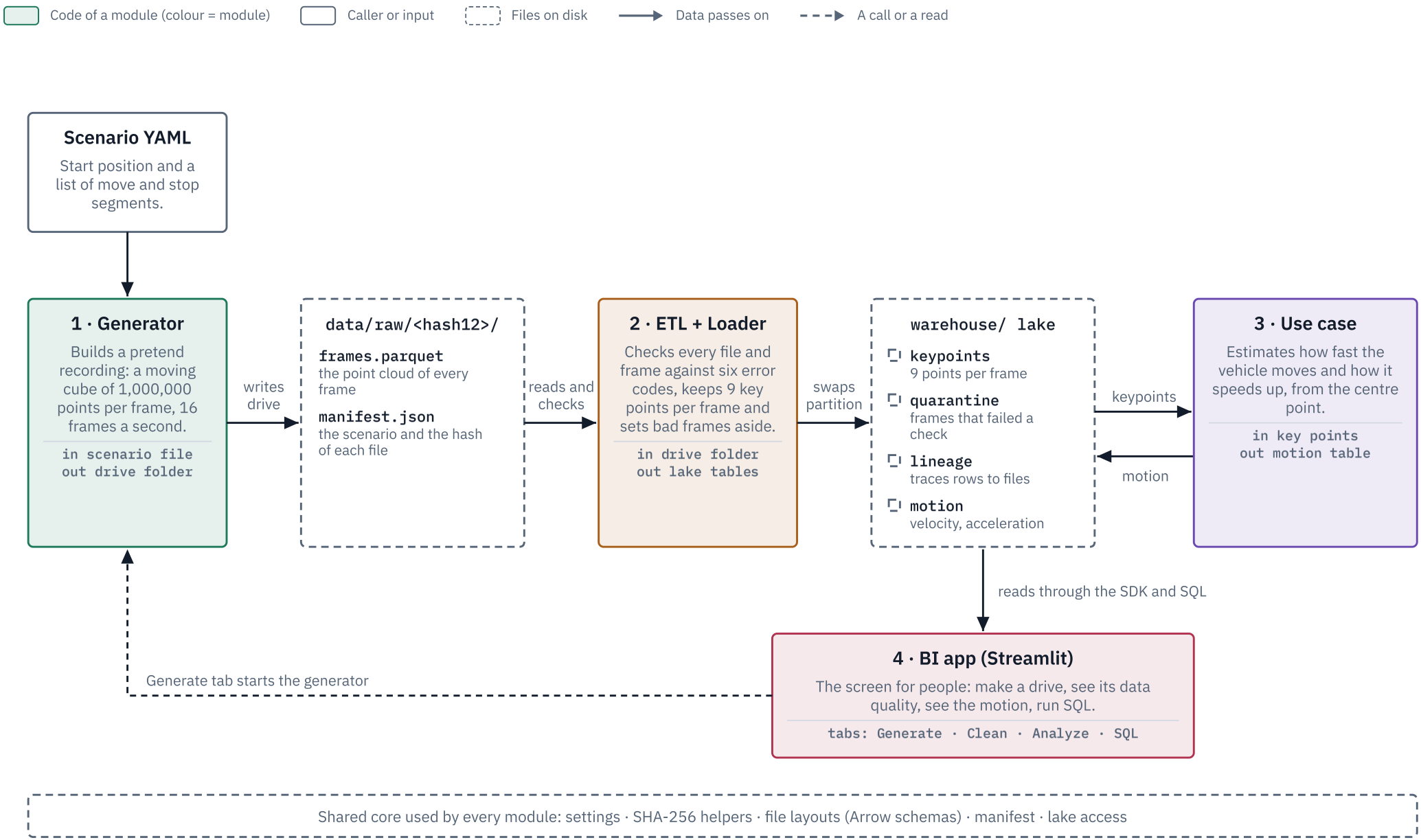
Window fit, not frame differences. COMPUTED
Positions are rounded to whole centimetres, so each can be off by 0.5 cm. A second difference at 16 fps multiplies that by 16 x 16 = 256, so two such errors can swing the acceleration by up to 512 cm/s², while the scenarios accelerate at about 100 cm/s². Velocity and acceleration therefore come from a sliding polynomial fit (16 frames, order 2).

The generator is tested by its inverse. DECISION
Property tests check that recovering a scenario from generated frames gives the scenario back, within measured tolerances.

Quality gates. MEASURED
pytest with hypothesis, ruff, mypy, Python 3.11. Module 1: 329 tests passed and 1 skipped on 26 Oct 2026; coverage was 100% at its last review gate, not re-run that day. Every assumption has a numbered entry in `docs/ASSUMPTIONS.md`.

System flow. Read left to right: a scenario file becomes a drive folder, the drive folder becomes lake tables, and the lake tables become a motion table. Each box says what it does and what goes in and out. The coloured squares in the lake show which module writes each table: orange for Module 2, violet for Module 3. The Generate tab is Module 1's screen, hosted inside the Module 4 app. The diagram is on the next page, turned sideways.

System flow


Code of a module (colour = module)
Caller or input
Files on disk
Data passes on
A call or a read
Scenario YAML
Start position and a list of move and stop segments.
1 · Generator
Builds a pretend recording: a moving cube of 1,000,000 points per frame, 16 frames a second.
in scenario file
out drive folder
writes drive
data/raw/<hash12>/
frames.parquet
the point cloud of every frame
manifest.json
the scenario and the hash of each file
reads and checks
2 · ETL + Loader
Checks every file and frame against six error codes, keeps 9 key points per frame and sets bad frames aside.
in drive folder
out lake tables
swaps partition
warehouse/ lake
keypoints
9 points per frame
quarantine
frames that failed a check
lineage
traces rows to files
motion
velocity, acceleration
keypoints
motion
3 · Use case
Estimates how fast the vehicle moves and how it speeds up, from the centre point.
in key points
out motion table
reads through the SDK and SQL
4 · BI app (Streamlit)
The screen for people: make a drive, see its data quality, see the motion, run SQL.
tabs: Generate · Clean · Analyze · SQL
Generate tab starts the generator
Shared core used by every module: settings · SHA-256 helpers · file layouts (Arrow schemas) · manifest · lake access

File structure

Every tracked source, config, doc and tool file. Module 1 comes from main. Modules 2 to 4 come from their branches, which are not merged into main yet, so the tree here is the union of the four.

Project root /

`.streamlit/config.toml`	Pins the Streamlit server to 127.0.0.1, because the SQL tab has no login.
`.gitignore`	Keeps the assignment PDF, generated data (data/, warehouse/, *.parquet), local environments and LLM-usage logs out of git.
`README.md`	Project readme. Holds only the title so far.
`pyproject.toml`	Package definition (vl-data, Python 3.11+): dependencies and settings for pytest, ruff and mypy.

config/ config/

`default.yaml`	Shared defaults: 16 fps, raw data folder, cube of 100 cm with 100 points per axis, round-trip tolerances of the generator.
`etl.yaml`	Loader settings: accepted schema versions, read chunk size, size limits, lake folder (VL_LAKE_DIR overrides), top speed of 9,000 cm/s, keypoint list.
`motion.yaml`	Motion estimator settings: 16-frame window, polynomial order 2, minimum points and gap limits.

docs/ docs/

`architecture_context.txt`	The agreed design decisions and the checklist of what the assignment asks for. The base of every module plan.
`ARCHITECTURE.md`	Design document: one section per module, with the data flow and the reason for each technology choice.
`ASSUMPTIONS.md`	Numbered log of every assumption and decision, with the reason and the consequence.
`ERROR_CODES.md`	List of error codes VL-001 to VL-006. Generated from the rule registry; a test fails if the file drifts.
`RELEASE_NOTES.md`	Product description, usage with commands, known issues, coming soon.
`SUPPORTING_DOCUMENTATION.md`	Generated single file: error codes, architecture and release notes in the order the assignment lists them.
`USE_CASES.md`	Six candidate use cases for the signal. One (ego-motion) is implemented.
`USE_CASE_ERRORS.md`	Named failures of the use case (MotionError, SQL refusals) and what to do about each.

tools/ tools/

`build_supporting_docs.py`	Joins ERROR_CODES, ARCHITECTURE and RELEASE_NOTES into SUPPORTING_DOCUMENTATION.md.
`motion_sweep.py`	Measures the motion estimator on generated drives to pick the window and order, and writes config/motion.yaml. Under review.

Package entry src/vl_data/

`__init__.py`	Package marker.
`__main__.py`	Entry point for python -m vl_data.
`cli.py`	Root command-line app (Typer). Mounts the commands of each module.

Shared core src/vl_data/core/

`config.py`	Configuration models and the YAML loader (cm, cm/s, frames). Checked on load.
`hashing.py`	Canonical JSON and SHA-256 helpers.
`schemas.py`	Arrow schemas of the drive files and the byte encoding of a point cloud.
`manifest.py`	Builds, writes, reads and verifies manifest.json (scenario hash plus the hash of every file).
`lake.py`	Hive-style Parquet lake: read, list, and replace the partition of one drive.
`lake_schemas.py`	Arrow schemas of the lake tables: keypoints, quarantine, lineage.

Module 1: generator src/vl_data/generator/

`models.py`	Scenario model: start state plus a list of move and stop segments. Strict validation.
`body.py`	The 1 m cube as a point lattice, and the indices of its corners.
`motion.py`	Position, speed and acceleration of every frame along x, from the segments.
`drive.py`	In-memory drive: the frames table.
`writer.py`	Writes a drive to disk in batches (zstd), checks it, then swaps the folder into place.
`faults.py`	Fault injection: drop frames, corrupt a frame, shift timestamps. Used to test the ETL.
`inverse.py`	Inverse of the generator: recovers track and segments from frames. Used by roundtrip.
`cli.py`	Commands generate, verify and roundtrip.
`scenarios/demo.yaml`	The 1,000-frame demo scenario (hash bf394b9bfb42).
`ui/generate_tab.py`	Generate tab: start form, segment table, speed preview, Generate button with size warning and disk-space check.
`ui/generate_app.py`	Standalone runner for the Generate tab (streamlit run).

Module 2: ETL and loader src/vl_data/etl/

`config.py`	Loader configuration models and loader for etl.yaml.
`body.py`	Reads the cube geometry from the manifest of a drive.
`keypoints.py`	Picks the 8 corners and the centre of the cube from a point cloud, by index.

facts.py	The single pass over the frames: one small fact row and the keypoints of each frame.
rules.py	Rule registry VL-001 to VL-006 and the runner that applies it.
ingest.py	Ingests one drive: verify hashes, validate, reduce to keypoints, swap the tables into the lake.
errors_doc.py	Builds docs/ERROR_CODES.md from the rule registry.
cli.py	Commands ingest and error-codes.

Module 3: read API and use case `src/vl_data/sdk/ and src/vl_data/usecase/`

sdk/client.py	VLClient: read-only access to the lake tables as Arrow tables, plus SQL (DuckDB).
usecase/config.py	Models of motion.yaml.
usecase/estimator.py	Sliding-window polynomial fit that gives velocity and acceleration from positions.
usecase/pipeline.py	Computes per-frame motion of the centre point and stores the motion table in the lake.
usecase/cli.py	Command motion.

Module 4: BI app `src/vl_data/bi/`

data.py	Data layer: drive list, quality numbers, commands and queries. No Streamlit.
figures.py	Plotly figures.
clean_tab.py	Clean tab: quality of one drive, kept and quarantined frames by error code. Read-only.
analyze_tab.py	Analyze tab: the stored position, velocity and acceleration against time.
sql_tab.py	SQL tab: one read-only SELECT over the lake views, example queries, 10,000-row display cap.
shell.py	App shell: drive selector in the sidebar, four tabs, scenario-hash badge.
app.py	Runner: python -m streamlit run src/vl_data/bi/app.py.

Tests `tests/`

conftest.py, test_cli.py	Shared fixtures and tests of the root command-line app.
core/ (5 test files)	Config, hashing, schemas, manifest, body limits.
generator/ (9 test files)	Scenario model, body, motion, writer and full density, faults, inverse and round trip, CLI, Generate tab.
etl/ (9 test files)	Config, body, keypoints, facts, each rule, ingest, error-code document, CLI, end to end.
sdk/ and usecase/ test files	VLClient; config, estimator, pipeline, motion sweep, CLI, end to end.
bi/ (5 test files)	Data layer, figures, SQL tab, tabs, shell.

Not listed: generated data (`data/`, `warehouse/`), the assignment PDF and the local LLM-usage files. All are git-ignored on purpose.

Module 1 · Generator

Builds the mock source. Given a scenario, it produces a drive: the full point cloud of a moving 1 m cube for every frame, and a manifest that lets anyone check the files. It does not read any other module's data. The other modules treat it as a black box that writes one folder.

IN

A scenario YAML: a start state (x, y, z in cm, speed in cm/s) and a list of move and stop segments. Settings from `config/default.yaml`: 16 fps, cube of 100 cm, 100 points per axis.

OUT

A drive folder `data/raw/<hash12>/`. `frames.parquet`: one row per frame, 1,000,000 points in one binary column. `manifest.json`: the scenario, the cube, fps, counts, schema version 2, and the hash and size of each file.

USED BY

Module 2 reads the frames and the manifest, and uses the fault injector in its tests. Module 4 hosts the Generate tab.

Module 1 structure. Read the arrows as “calls”. The Generate tab and the command line (generate) both build a Scenario with models.py and then call generate_to_disk in writer.py. The writer calls iter_frame_batches in drive.py, which uses motion.py and body.py, and writes the drive folder. So the two callers run the same pipeline; the motion and drive steps happen inside generate_to_disk, not in the callers. The tab does more before it writes: it checks the size and hash, skips a drive that already exists, checks the free disk space and warns about the size, and it draws a speed preview with motion.py. The command line loads the file and writes. It also has two other commands: roundtrip, which calls the inverse (dashed), and verify, which calls the manifest check of the shared core (dashed, right edge). write_drive, the other function in writer.py, is used by tests and for faulty drives. faults.py is called by tests only. The diagram is in the appendix at the end of this report (Diagram 1, turned sideways).

How a drive is made

1. **Load and validate the scenario**
   A YAML file with a start state and a list of move and stop segments. A stop needs the vehicle at rest, durations must be whole frames, and a scenario has at most 2,000 frames.
2. **Hash it**
   SHA-256 of the canonical JSON of the scenario and the body (size and points per axis). The first 12 hex characters name the drive folder.
3. **Build the motion**
   A straight-line ramp of speed inside each segment, integrated into a position per frame.
4. **Build the frames**
   The 100 x 100 x 100 lattice, shifted by the rounded position, in batches of 50 frames so memory stays near one batch.
5. **Write and check**
   The frames go to a temporary folder, the manifest is written last, the folder is swapped into place, and the drive is verified.

Fault injection

The generator can damage a drive on purpose so that Module 2 can be tested against known errors. `inject_faults` returns a new drive and a count per error code. Only the frames table changes; the scenario hash is copied unchanged. A faulty drive still passes `verify_drive`, because its manifest is built from the faulty file.

FAULT	TRIGGERS	WHAT IT DOES TO THE FRAMES TABLE
DropFrame	VL-001	Deletes that frame's row. Other frame ids are not renumbered.
ShiftTimestamp	VL-002	Adds a shift to one timestamp. The shift must be smaller than one frame interval.
DuplicateFrame	VL-003	Inserts a copy of the row right after the original.
CorruptPoints	VL-004	Removes the last point (12 bytes) from one frame.
BreakSchema	VL-005	Renames the column points to pts.
Teleport	VL-006	Adds a distance to the x coordinate of every point of one frame.

Module 2 · ETL + Loader

Takes the drive folder Module 1 wrote and turns it into clean, queryable tables. It verifies every file, tests each frame against six error codes, keeps nine key points per frame, and replaces the drive's rows in a Parquet lake. Running it twice gives the same result.

IN

A drive folder `data/raw/<hash12>/` with `frames.parquet` and `manifest.json`. Settings from `config/etl.yaml`: the nine key points, the speed limit of 9,000 cm/s, the read chunk size.

OUT

Three lake tables, one folder per drive. `keypoints`: frame, timestamp, key point name, x, y, z. `quarantine`: rejected frames or problems, with an error code and a message. `lineage`: one row per drive with its status (ok or rejected), counts of frames expected, in, kept, rejected and missing, and the hash of the frames file.

USED BY

Module 3 reads the keypoints and lineage through VLClient. Module 4 reads quarantine and lineage for the Clean tab and queries all of them in the SQL tab.

Module 2 structure. The numbered boxes are the main line: (1) check the drive and its hashes, (2) scan the frames once and pick the key points, (3) apply the rules, (4) swap three tables into the lake. body.py and config.py feed the steps with the cube size and the settings. The command line starts ingest, and error-codes asks errors_doc.py to write the error-code document from the rule list. The lake and the manifest check come from the shared core. The diagram is in the appendix at the end of this report (Diagram 2, turned sideways).

How a drive is ingested

1. **Gates, cheapest first**
   The folder name must be a valid drive id. The manifest must read, its folder name must equal the first 12 hex characters of the scenario hash, the schema version must be supported, and the frame rate must divide a second. Only then is every file hash checked, the slow step. Any failure rejects the whole drive before a frame is read.
2. **Schema gate**
   Open frames.parquet, refuse any row group above the size cap, and compare the columns with the agreed frames schema (VL-005).
3. **One scan**
   Read the frames file once, in chunks. For each frame keep a small record: frame number, timestamp, payload size, payload hash, whether the points form the expected cube, and the centre. For each valid frame also pick the 9 key points, so the file is never read twice.
4. **Rules**
   Run VL-001 to VL-004 on the records. Run VL-006 only on frames that passed those, starting from a trusted frame so one teleported frame cannot poison the rest.
5. **Reduce**
   Keep the key-point rows of the accepted frames, sorted by frame number and then by the key-point list, and write them as one batch. The file bytes do not depend on the chunk size.
6. **Write**
   Write each table's new partition aside, then swap it in for the old one: keypoints, quarantine, then lineage. Lineage goes last and acts as the commit marker. A rejected drive drops its keypoints and gets a quarantine row and a lineage row with status rejected.

Module 3 · Use case

The use case: the velocity and acceleration of the vehicle. It reads the centre point of every frame from the lake, fits a sliding window on the real timestamps, and stores the result as a motion table. It also owns VLClient, the read-only way into the lake that Module 4 uses too.

IN

The lake tables `keypoints` and `lineage`, read through VLClient, and `config/motion.yaml`: a window of 16 frames, polynomial order 2, at least 9 valid points in a window, gaps up to 4 frames.

OUT

The `motion` table, one row per frame: frame, timestamp, position x, velocity, acceleration (null where the window is too sparse) and the number of points in the window. One partition per drive.

USED BY

Module 4 draws the table in the Analyze tab and queries it in the SQL tab. Anyone can query it with SQL through VLClient.

Module 3 structure. The numbered boxes are the main line: (1) VLClient reads the centre-point rows, (2) the pipeline checks the drive and prepares the data, (3) the estimator fits the sliding window, (4) the motion table is swapped into the lake. The settings come from config/motion.yaml. Module 4 and any SQL user read the stored table back through the same VLClient. The diagram is in the appendix at the end of this report (Diagram 3, turned sideways).

How motion is computed

1. **Check the drive**
   The drive must have exactly one lineage row, and its status must be ok. Otherwise the command stops with a named error and writes nothing.

2. **Read the centre point**
   Through VLClient, take the rows of the centroid key point for the drive: one position per frame, with its real timestamp.
3. **Fit a window around each frame**
   Around every frame take a window of 16 frames on the real timestamps, so a missing frame is a gap and not a shifted sample. Fit a polynomial of order 2 by least squares. A window needs at least 9 valid points and no gap wider than 4 frames.
4. **Read off the motion**
   The fitted curve gives the position, its slope is the velocity and its curvature is the acceleration. A frame whose window is too sparse gets a null velocity and acceleration.
5. **Store it**
   Replace the drive's motion partition in the lake in one swap. A second run leaves the file byte for byte the same.

The motion table

COLUMN	TYPE	MEANING
`frame_id`	int64	Frame number.
`timestamp_ns`	int64	Time of the frame in nanoseconds.
`x_cm`	float64	Fitted position of the centre along x, in cm.
`velocity_cm_s`	float64, null allowed	Signed velocity in cm/s. Null where there is no estimate.
`acceleration_cm_s2`	float64, null allowed	Signed acceleration in cm/s². Null likewise.
`n_points`	int32	How many points the window held.

VLClient, the read-only SDK

The one way Modules 3 and 4 read the lake. A valid drive that is not in the lake gives an empty table; an invalid drive id raises an error.

METHOD	WHAT IT RETURNS
`drives()`	List the drives that have key points.
`keypoints(drive, keypoint=None)`	Key points of a drive, ordered by frame and key point name.
`motion(drive)`	The motion table of a drive.
`lineage(drive)`	The lineage row of a drive.
`quarantine(drive)`	The rejected frames and problems of a drive.
`sql(query)`	Run one SELECT over the table views with DuckDB. Closes its connection after every query.

Module 4 · BI

One Streamlit app that lets a person look at a drive: generate one, see its data quality, see its motion, and ask the lake their own question in SQL. It reads everything through VLClient and writes nothing.

IN
The lake through VLClient (key points, quarantine, lineage, motion, SQL) and a drive selector in the sidebar. The Generate tab is Module 1's screen.

OUT
Screens only. The app writes nothing to the lake. When a drive is missing data it prints the exact ingest or motion command and does not run it.

USED BY
People: a data scientist exploring one drive at a time.

Module 4 structure. app.py starts shell.py, which draws the sidebar and the four tabs. The Generate tab is Module 1's and writes a drive folder. The Clean, Analyze and SQL tabs use data.py (no Streamlit code) and figures.py (the charts); the Clean and Analyze tabs use both, the SQL tab uses data.py. Everything is read through VLClient, which reads the lake tables. The diagram is in the appendix at the end of this report (Diagram 4, turned sideways).

The four tabs

A sidebar selector picks the drive. Every tab shows the scenario hash of that drive as a badge. The app adds no error codes: the Clean tab shows VL-001 to VL-006 and INTEGRITY.

TAB	WHAT IT SHOWS	READS
Generate	A form for a scenario, a speed preview and a Generate button. Writes a drive folder to disk. This is Module 1's screen, unchanged.	Module 1 writer
Clean	For the selected drive: frames expected, in, kept, rejected and missing, a bar chart of kept, rejected and missing frames, a bar chart of quarantine rows per error code, and the quarantine table (up to 200 rows, with the full count stated). A drive that was never ingested gets the exact ingest command to run; the app does not run it.	lineage, quarantine
Analyze	The stored motion: position, velocity and acceleration against time. A drive that has been ingested but has no motion yet gets the exact motion command to run.	motion
SQL	One read-only SELECT over the views keypoints, quarantine, lineage and motion, with five example queries in a dropdown. The result shows up to 10,000 rows and states the total. A non-SELECT, a second statement or a bad query shows a one-line message.	all four tables

How it is checked

From the Module 4 documents.

The tabs are tested with Streamlit's test runner on a small drive (a 100 cm cube of 10 points per axis). A live server was started once by a script: its health check, the page and the run of `app.py` answered, and the server log had no exception. That shows the app starts and is served. It does not show what a browser draws, and it does not cover any click.

Supporting documentation

The assignment asks for supporting documentation: the list of error codes, the architecture and design, and release notes. This tab holds all three, plus the use cases. The table shows where each requirement is covered in this report.

WHAT THE ASSIGNMENT ASKS FOR	WHERE IT IS IN THIS REPORT
At least three use cases that could benefit from the product	Use cases (below): six, one implemented
A mock data source; not all the data has to be saved	Module 1 tab. The demo drive has 1,000 frames
An ETL that converts the data for the chosen use case	Module 2 tab (loader and ETL) and Module 3 tab (the use-case algorithm)
A data warehouse with a generic structure for user queries	Module 2 tab (lake tables) and Architecture and design: data model
A BI view that presents the solution	Module 4 tab
A reason and the alternatives for each technology	The design-choices table in Architecture and design (the web page also has one per module tab)
Supporting documentation: the list of possible error codes	Error codes (below)
Supporting documentation: architecture and design	Overview, the four module tabs, and Architecture and design (below)
Supporting documentation: release notes with product description, usage example, known issues and coming soon	Release notes (below)
LLM usage: the model, the main prompts and the approach	A separate document, docs/LLM_USAGE.md, kept out of git on purpose and sent with the submission. It is not part of this report.

Use cases

Six use cases were considered. One is implemented.

#	USE CASE	WHO BENEFITS	WHAT IT NEEDS FROM THE SIGNAL	STATUS
1	**Ego-motion extraction: velocity and acceleration**	Algorithm teams that validate motion estimates	Position per frame; velocity and acceleration follow from it	**Implemented**
2	Search for driving behaviour (hard braking, reversing, stopping)	Teams building training sets from rare moments	Velocity and acceleration over time, plus rules that mark events	Considered; outside this version
3	Training-data mining for detectors	Detection teams that need hard, rare moments	Event time ranges that a clip service can cut	Considered
4	Detection quality sliced by driving condition	Teams testing detection models, who ask whether a model gets worse while braking or reversing	A motion-condition label for every frame	Considered
5	Localization quality and drift	Map and localization teams comparing repeated drives of one route	Trajectories per drive	Considered
6	Dataset balancing and train/validation splits	Data teams avoiding near-identical drives on both sides of a split	Drive keys and motion summaries	Considered

WHY USE CASE 1

- The warehouse stays generic. A position per frame is useful to every other use case in the table, so the stored tables do not depend on this one.
- It is the base of the others. Use cases 2 to 6 all start from velocity and acceleration.

Error codes

The six rule codes of the loader, the drive-level code, and the failures of the other modules.

CODE	NAME	WHAT IT MEANS	HOW IT IS DETECTED	FAULT USED IN TESTS
VL-001	Missing frame	A frame_id between 0 and the expected count minus 1 has no row in the frames file.	Every id in the expected range that is absent from the frame_id column.	`DropFrame(frame_id)`
VL-002	Timestamp gap	A frame's timestamp is not frame_id times the frame interval.	timestamp_ns differs from frame_id x frame interval, checked per row.	`ShiftTimestamp(frame_id, delta_ns)`
VL-003	Duplicate frame	More than one row carries the same frame_id.	Rows are grouped by frame_id. If all rows of a group have the same timestamp and payload hash, the first is kept and the repeats are flagged; otherwise every row of the group is flagged.	`DuplicateFrame(frame_id)`
VL-004	Corrupt point cloud	A frame's points are not the expected body: wrong length, count or shape.	The payload must be a multiple of 12 bytes, decode to the drive's point count, and each axis of the offset from the expected lattice may spread by at most 1 cm.	`CorruptPoints(frame_id)`
VL-005	Schema mismatch	The frames file does not have the columns and types of the frames schema.	Rejects the whole drive. The file schema (metadata ignored) is compared with the frames schema.	`BreakSchema()`
VL-006	Impossible motion	The body moved farther between frames than the speed limit allows.	Among frames with no other violation, in frame order, a centre farther from the last accepted centre than max speed x frame gap / fps + margin is flagged and never becomes the baseline.	`Teleport(frame_id, dx_cm)`

DRIVE-LEVEL PROBLEMS: INTEGRITY

These are not frame errors, so they have no VL code. Each is written to the quarantine table as a drive-level row with the code INTEGRITY, and the drive is marked rejected in the lineage table. At most 20 problems are stored per drive. The causes:

- An unreadable manifest: missing, not valid JSON, invalid, or a schema-1 manifest without a body.
- A folder name that is not the first 12 hex characters of the scenario hash, or a malformed scenario hash.
- An unsupported schema version, or a frame rate below 1 or one that does not divide a second evenly.
- frames.parquet not listed in the manifest, or more points per frame than the configured limit.
- A changed, missing or extra file found by the manifest check.
- An unreadable or implausible frames file: not Parquet, a footer that claims too many rows, or a row group above the size limit.
- A null frame number or timestamp, or a frame number outside 0 to the expected count minus 1.

USE-CASE FAILURES (MODULE 3, COMMAND MOTION)

These have no VL code. They describe a drive that cannot be processed. Nothing is written when one is raised; an old motion table stays as it was.

WHEN IT HAPPENS	WHAT TO DO
The drive has no lineage row (never ingested, or a wrong id).	Run ingest on the drive folder; check the 12-character id.
The lineage table has more than one row for the drive.	Re-ingest the drive; ingest replaces the lineage partition.
The lineage status is not ok (the drive was rejected).	Read the quarantine table for the reason, fix the raw drive, ingest again.
The drive has no rows for the position key point (default: centroid).	Check the name against the key points in config/etl.yaml.
The timestamps of the key-point rows are not strictly increasing, or an x value is not finite.	Re-ingest the drive.

OTHER FAILURES

- A scenario is rejected by the generator before any file is written: a duration that is not a whole number of frames, a move whose start and end speed are both 0, a stop while the vehicle is moving, or more than 2,000 frames.
- VLClient.sql raises ValueError when the text cannot be parsed, holds more than one statement, or is not a SELECT. A drive id with a character other than letters, digits, _ and - raises ValueError.

EXIT CODES OF THE COMMANDS INGEST AND MOTION

CODE	MEANING
0	Done. Quarantined frames are normal.
1	Refused or failed: a drive was rejected (ingest), or MotionError or an unexpected failure (motion).
2	Usage or configuration error, for example a missing lake folder.
3	A file could not be read or written.

Architecture and design

The system flow, the file structure and the reasons for each module are in the Overview and the four module tabs. This section adds the data model, the project-wide design choices with their alternatives, the storage arithmetic, and what a production version would change.

DATA MODEL

TABLE OR FILE	LAYER	CONTENT
frames.parquet	raw drive	frame_id, timestamp_ns, points (1,000,000 x 3 int32 = 12,000,000 bytes at the shipped density, one binary column)
manifest.json	raw drive	schema version 2, drive hash, scenario, body (size_cm, points_per_axis), fps, counts, SHA-256 and size of each file
keypoints	lake	frame_id, timestamp_ns, keypoint, x_cm, y_cm, z_cm: 9 rows per kept frame
quarantine	lake	frame_id, row_index, code, message: one row per rejected frame or drive-level problem
lineage	lake	scenario_hash, schema_version, status, frames expected / in / kept / rejected / missing, keypoint_rows, frames_sha256: one row per drive
motion	lake	frame_id, timestamp_ns, x_cm, velocity_cm_s, acceleration_cm_s2, n_points: one row per frame

The lake tables use a generic shape that does not depend on the use case: a drive and frame key, a timestamp, and numeric measures.

DESIGN CHOICES AND ALTERNATIVES

DECISION	CHOICE	WHY	ALTERNATIVES	WHEN TO CHANGE IT
Motion model	Straight line along x, signed speed, no rotation	A cube's orientation cannot be read from its points	Planar motion; full rotation with a non-symmetric body	The real signal rotates
Estimator	A least-squares sliding window on the real timestamps	Positions are rounded to whole cm, so a second difference at 16 fps can swing by up to 512 cm/s² (computed) while scenarios accelerate at about 100 cm/s²	Per-frame differences; Savitzky-Golay; a Kalman smoother	Noisy data or rotation
Key points	8 corners and the centre, picked by lattice index	The corner spacing doubles as a rigid-shape check (VL-004)	Track one reference point; keep every point	Rotation or a changing point set
Read access	A read-only Python SDK (VLClient) returning Arrow tables, plus SQL	Exact column types and real nulls; no write path from the SDK	pandas DataFrames; a web service (coming soon)	Many users
Position estimate	Centroid of a frame's points	Exact for pure translation; needs no matching between frames	Tracking one reference point	Rotation or a changing point set
Body	1 m cube, 100 x 100 x 100 = 1,000,000 points	Matches the example density of the assignment; fewer frames are allowed, thinner frames are not	A 10 x 10 x 10 sample (the earlier choice); a car-sized box	Storage or time limits
Source	The generator writes a drive to disk	The simplest valid source	A service or a stream	Many consumers or live data

DECISION	CHOICE	WHY	ALTERNATIVES	WHEN TO CHANGE IT
Config	YAML plus a validating Python loader	Readable; fails early on bad values	Environment variables only; JSON	Many deployment environments
Scenario validation	pydantic	Typed models with clear errors	Dataclasses plus hand checks	Not needed so far
Hashing	SHA-256 over canonical JSON (scenario) and file bytes (files)	Standard; checkable with common tools	Content hashes; UUIDs	Needing stability across library versions
Raw file format	Parquet via PyArrow, one row per frame, zstd	Columnar, compact, readable by every tool	One row per point; CSV; HDF5	Per-point SQL
Query	DuckDB over Parquet folders	SQL with no server	Trino; Athena	Many users, large data
ETL engine	pandas and numpy	The data is small and the code is clear	Polars; Spark	Data no longer fits one machine
Lake format	Plain Parquet, Hive-style folders, behind one class	Nothing to run	Apache Iceberg; Delta Lake	Many writers, schema changes, time travel
Idempotent writes	Write to a temporary folder, then swap the drive folder	Re-running never duplicates; a crash never loses old data	Blind append; upsert	Concurrent writers
Validation	A rule registry with error codes	Explicit and testable	Great Expectations; pandera	Many datasets and owners
BI	Streamlit and Plotly	One Python file per view	Superset; Grafana	Non-Python users
Command line	Typer	Typed commands	argparse	None
Tests	pytest and hypothesis	Property tests suit a two-way check	unittest	None
Not used	Kafka, Spark, Airflow, dlt, Docker	No purpose at this scale	See scaling	See scaling

STORAGE ARITHMETIC

A full-density frame is 100 x 100 x 100 = 1,000,000 points. At 3 coordinates of 4 bytes each that is 12 MB per frame, 192 MB per second at 16 fps, and about 691 GB per hour (computed). The 1,000-frame demo is therefore 12 GB before compression (computed) and 287,134,433 bytes on disk with zstd (measured), about 42 times smaller (computed). A production design would store a keyframe and the ego-motion change per frame instead of every point.

SCALING TO PRODUCTION

Apache Iceberg or Delta on object storage with a managed catalog, Spark or Flink for large drive counts, keyframe-plus-delta storage, schema contracts between producer and consumers, partitioning and compaction, and scheduled runs with an orchestrator.

LIMITS OF THIS DESIGN

- No rotation. A real vehicle rotates, and reading its orientation needs a non-symmetric body or tracked points.
- Plain Parquet has no atomic commit across files, no schema-evolution tracking and no time travel. The folder swap protects one drive at a time and is not safe for concurrent writers.
- File hashes depend on the pyarrow version that wrote the files.
- Single machine, CPU only.

Release notes: version 0.1.0

Numbers are from real runs of the project unless a line says it is reported by another session or comes from reading the code.

PRODUCT DESCRIPTION

A small data product for a vehicle-location signal. It generates a mock drive from a start position and a motion profile, stores the point cloud, checks it and reduces it to key points, and extracts the vehicle's velocity and acceleration from the points alone. The vehicle is a 1 m cube that moves in a straight line, forwards, backwards and stopped, at different speeds. All data is synthetic.

WHAT IS IN THIS VERSION

PART	WHAT IT DOES	STATUS
Generator	Writes a hashed drive: a 16 fps point cloud and a manifest. Checks itself by recovering the scenario from the data.	Built
Loader / ETL	Verifies hashes, validates frames with error codes VL-001 to VL-006, keeps 9 key points per frame, and writes keypoints, quarantine and lineage tables to a queryable lake.	Built
Use-case algorithm	Computes velocity and acceleration from the stored centre point (motion), and serves the lake through the Python SDK (VLClient) and SQL.	Built
BI	One Streamlit app with four tabs: Generate, Clean, Analyze, SQL.	Built

INSTALL AND FIRST CHECK

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -e ".[dev]"
.venv\Scripts\python.exe -m vl_data --version
```

The last command prints vl_data 0.1.0 and exits with code 0. Python 3.11 or newer is required; only 3.12 has been tested. Run on 23 Oct 2026, Windows PowerShell, Python 3.12.6.

USAGE AND EXAMPLES

GENERATE THE DEMO DRIVE

```
python -m vl_data generate --scenario src/vl_data/generator/scenarios/demo.yaml --out data/raw
```

```
frames: 1000
timestamp_spacing_ns: 62500000
points_per_frame: 1000000
scenario_hash: bf394b9bfb421fda860ab97fbab54d5c36c995a661e770969cbffd9a2afa95c1
drive_dir: data\raw\bf394b9bfb42
manifest: verified
```

Measured on 24 Oct 2026: 81 s including verification. 1,000 frames hold 12 GB of points before compression; frames.parquet is 287,134,433 bytes with zstd. Generating the same scenario again replaces the folder with identical files.

CHECK A DRIVE AGAINST ITS MANIFEST

```
python -m vl_data verify data/raw/bf394b9bfb42
```

```
manifest: verified
```

Exit code 0. After one byte of a file was changed in a copy, it printed a line such as 'changed: <file> (sha256 mismatch)' on stderr and exited with code 1.

CHECK THAT THE GENERATOR CAN READ ITS OWN DRIVE BACK

```
python -m vl_data roundtrip --scenario src/vl_data/generator/scenarios/demo.yaml
```

```
segments: expected 8, found 8
max_boundary_error_frames: 4
max_duration_error_frames: 5
max_end_speed_error_cm_s: 24.4
round trip OK
```

Measured on 24 Oct 2026: 1.41 s, nothing is written to disk. The limits are 6 frames, 8 frames and 40 cm/s (config/default.yaml). The check is only tested for segments of 32 frames (2 s) or more.

LOAD THE DRIVE INTO THE LAKE

```
python -m vl_data ingest data/raw/<hash12>
python -m vl_data ingest --all data/raw --lake-dir warehouse
python -m vl_data error-codes --check
```

```
drive: bf394b9bfb42
status: ok
frames_expected: 1000
frames_in: 1000
frames_kept: 1000
frames_quarantined: 0
keypoint_rows: 9000
points_per_frame_in: 1000000
keypoints_per_frame_out: 9
```

Measured on 25 Oct 2026: about 44 s per drive on one machine (output shortened here). error-codes --check exits 0 when docs/ERROR_CODES.md matches the rule list.

COMPUTE VELOCITY AND ACCELERATION

```
python -m vl_data motion bf394b9bfb42 --lake-dir warehouse
```

```
drive: bf394b9bfb42
frames_read: 1000 frames
frames_estimated: 1000 frames
frames_null: 0 frames
```

Reported on 25 Oct 2026: about 1 s. Running it again leaves the stored file byte for byte the same.

QUERY THE MOTION TABLE FROM PYTHON

```
from vl_data.core.lake import Lake
from vl_data.sdk.client import VLClient

client = VLClient(Lake("<your lake folder>"))
client.sql("SELECT frame_id, round(velocity_cm_s, 1) AS velocity_cm_s FROM motion WHERE frame_id IN (0, 250, 500, 750, 999) ORDER BY frame_
```

```
[{'frame_id': 0, 'velocity_cm_s': 0.0}, {'frame_id': 250, 'velocity_cm_s': 600.0},
 {'frame_id': 500, 'velocity_cm_s': -300.1}, {'frame_id': 750, 'velocity_cm_s': -0.0},
 {'frame_id': 999, 'velocity_cm_s': 0.0}]
```

Measured on 25 Oct 2026 on the small test body (100 cm cube, 10 points per axis). sql() accepts one SELECT and returns an Arrow table.

OPEN THE BI APP

```
python -m streamlit run src/vl_data/bi/app.py --server.address 127.0.0.1 --server.headless true --browser.gatherUsageStats false
```

```
(a local web page with the tabs Generate, Clean, Analyze, SQL)
```

Run it from the repository root. --server.address 127.0.0.1 keeps the page on this computer, which matters because the SQL tab has no login. Checked by Streamlit's test runner and one live-server start; not opened in a browser by hand.

A SCENARIO FILE

```
seed: 0
start: {x_cm: 0, y_cm: 0, z_cm: 0, speed_cm_s: 0}
segments:
  - {type: stop, duration_s: 2.0}
  - {type: move, duration_s: 6.0, end_speed_cm_s: 600}
  - {type: move, duration_s: 8.0, end_speed_cm_s: -300}
  - {type: stop, duration_s: 4.0}
```

Speeds are cm/s and positive means forward. A move changes the speed in a straight line from the current speed to end_speed_cm_s. Durations must be a whole number of frames (a multiple of 1/16 s).

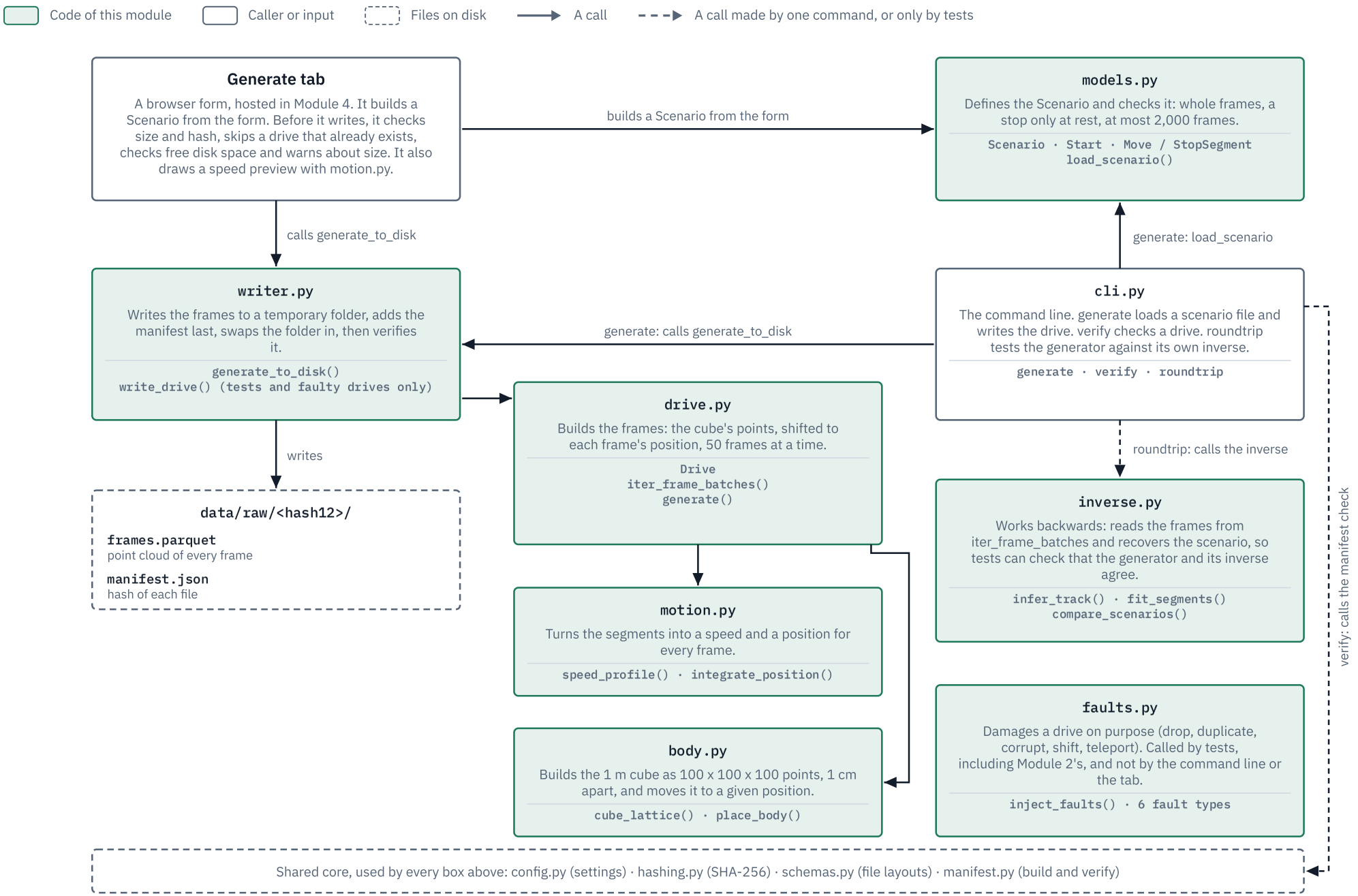
KNOWN ISSUES

- No rotation. The vehicle never turns or yaws. Motion is one-dimensional, along x.
- A drive at full density is large: 12 GB of points per 1,000 frames before compression (287,134,433 bytes on disk with zstd, measured). A scenario may have at most 2,000 frames (125 s at 16 fps).
- generate() and write_drive hold a whole drive in memory and refuse a drive over 1 GiB of points (more than 89 frames at full density). Use the generate command, which streams.
- Values are rounded to whole centimetres, which adds small errors. Per-frame acceleration is too noisy to use directly, so it is fitted over windows of 16 frames. The fit smooths speed changes, and the first and last 8 frames are less accurate.
- File hashes only match when the same pyarrow version wrote the files.
- Plain Parquet has no atomic multi-file commit, no schema-evolution tracking and no time travel. Loading a drive replaces its three tables one after another; a crash in between is repaired by loading the drive again. The lake is single-writer: load one drive at a time.
- The loader reads the frames file twice (once to hash it, once to scan it). Measured peak memory was about 1.7 to 1.9 GiB at full density on one machine.
- The two-way check (roundtrip) is only tested for segments of 32 frames (2 s) or more; the command warns for shorter ones.
- The loader's speed limit (9,000 cm/s, a Tesla's top speed rounded up) is set in a YAML file. A faster vehicle would be flagged as impossible motion. VL-006 checks position only; no rule checks acceleration.
- VL-004 allows for the 1 cm rounding wobble of whole-centimetre points, so a single point moved by exactly 1 cm is not detected; a move of 2 cm or more is.
- frames_rejected counts quarantine rows that name a file row, so a frame that breaks two rules counts twice. A drive that cannot be read is rejected with INTEGRITY, not with one of the six VL codes.
- The motion table is not refreshed by a new ingest. After re-ingesting a drive, run motion again.
- VLClient.sql runs any single SELECT, including read_parquet on any file the user can read. It is a local single-user tool; do not expose it to untrusted input. The BI app must stay on 127.0.0.1 because its SQL tab has no login.
- BI app: run it from the repository root, because the default lake and raw folders are relative to the current folder. The SQL box text is not refreshed when the selected drive changes, and the Generate tab's badge shows the hash of the drive selected in the sidebar.
- Single machine, CPU only.

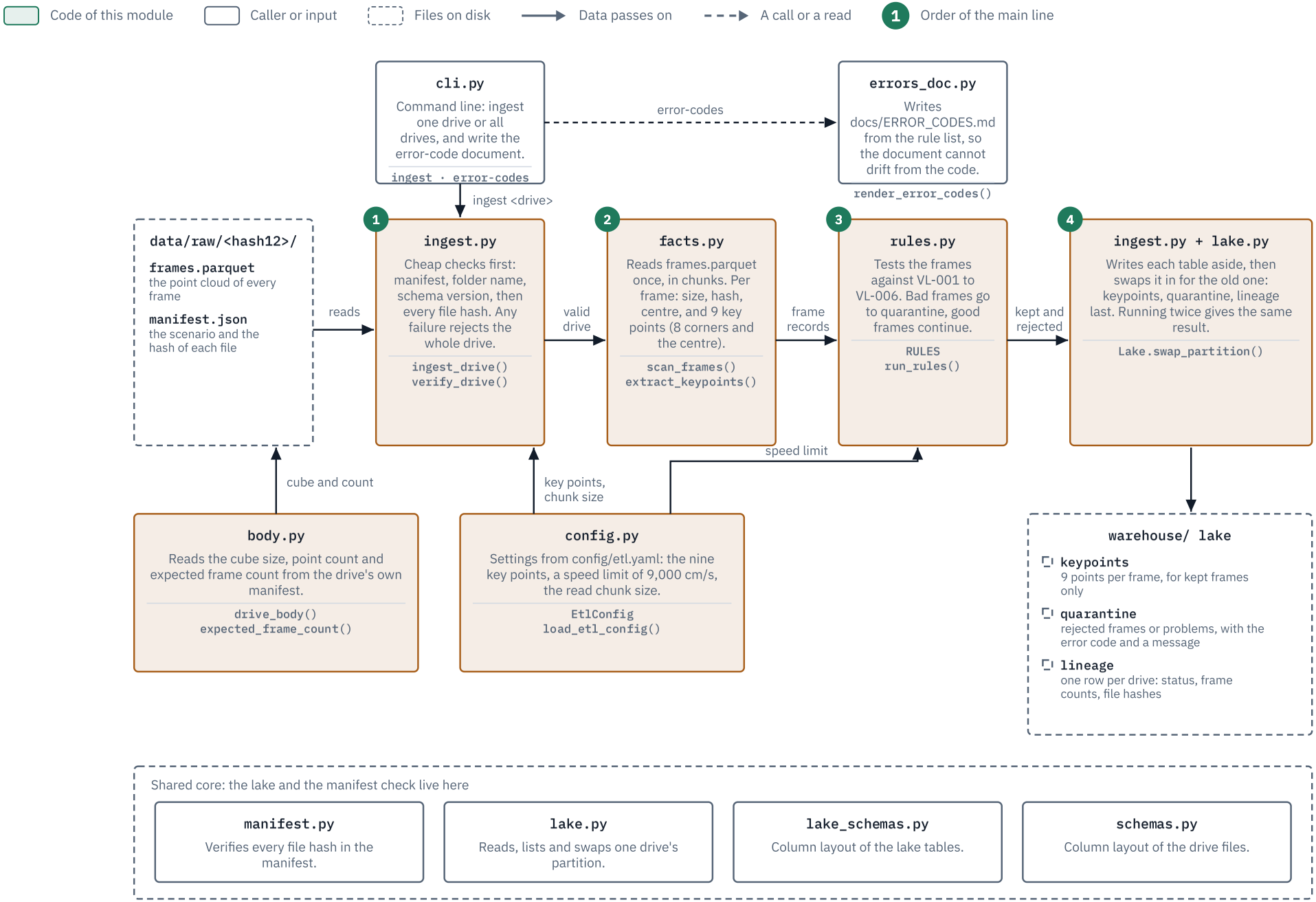
COMING SOON

- Rotation and yaw with a non-symmetric body, planar motion, optional noise, event detection and clip candidates.
- Apache Iceberg as the table format, a REST API over the SDK, streaming ingestion (for example Kafka), parallel loading of many drives, a Spark backend, an object-store profile, scheduled runs, and delta-encoded storage.
- A data-quality report function; a Kalman or RTS smoother as an alternative to the window fit.
- A locked SQL connection (external access off, extension autoload off, a memory limit) and a deeper link check on the raw folder.

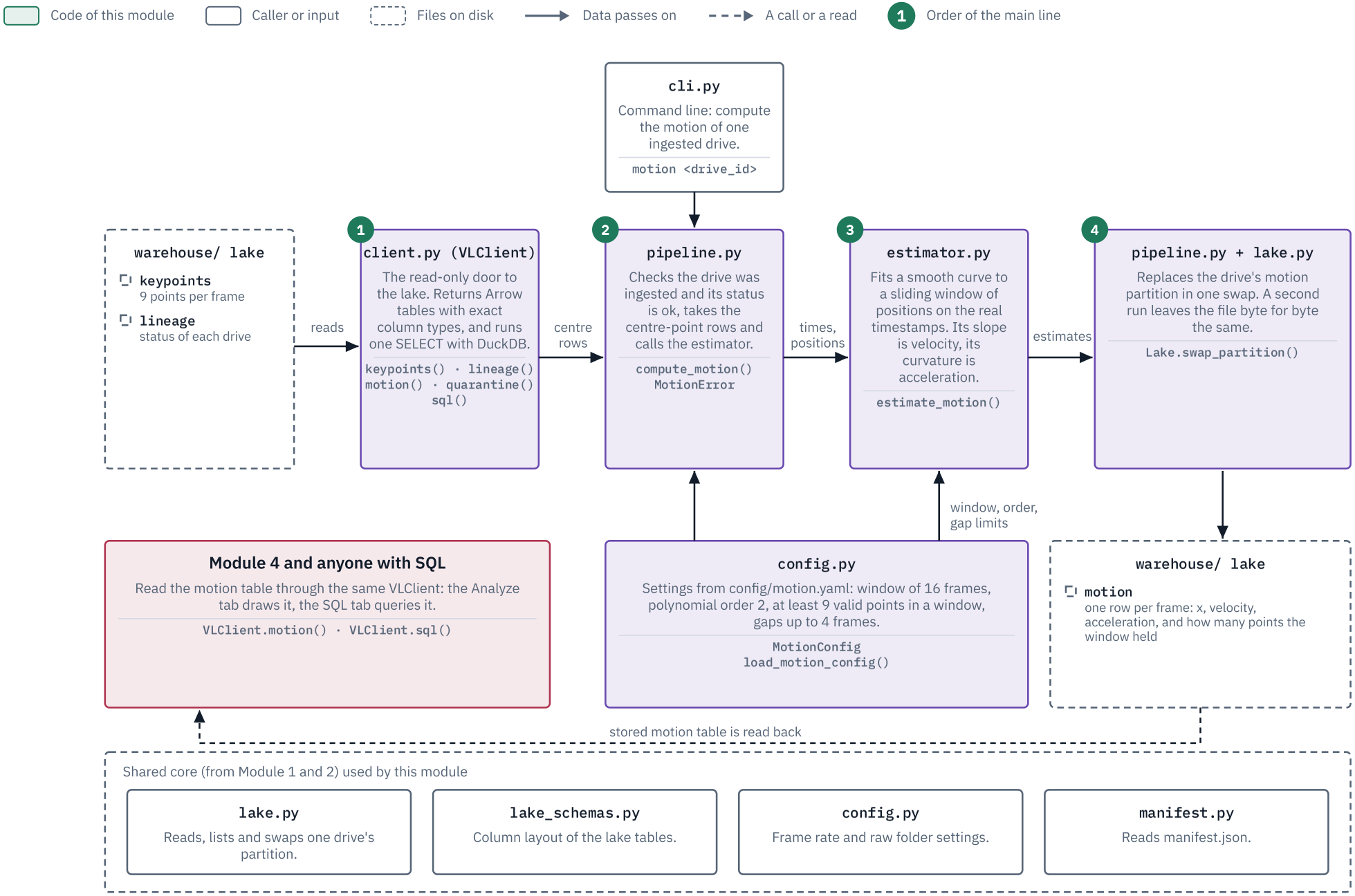
Module 1 structure



Module 2 structure



Module 3 structure



Module 4 structure

